MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

Milestone 2: Enhancing Functionality – User Features & Integrations

Prepared and Presented by:

Bigbig, Alvin Kent
Casusi, Shakira Angela
Mirasol, Shaira Nicole
Umali, Camille Rose
Villarasa, Franze
Bachelor of Science in Information Technology
Term 2 - A.Y 2025-2026

TABLE OF CONTENTS

1. System Overview

1.1. Purpose

The Connectly system is a backend service designed to provide a robust, scalable, and secure foundation for a social media application. Built with Django and the Django REST Framework, its primary purpose is to manage core social networking entities, including users, posts, and their interactions. The architecture emphasizes a clean RESTful API, clear separation of concerns, and secure, token-based authentication, making it a reliable engine for a client-facing application.

1.2. Summary of Added Features

This development milestone significantly enhances the system's social capabilities by introducing several key features:

Features	Explanation
Liking System	A `Like` model and corresponding API endpoint (`/posts/<id>/like/`) were implemented, allowing authenticated users to like a post. The system prevents duplicate likes from the same user on the same post.
Commenting System	The functionality for users to comment on posts was added. This includes a Comment model and API endpoints to create a comment on a specific post (`/posts/<id>/comment/`) and list all comments for that post

	(`/posts/<id>/comments/`).
Google OAuth 2.0 Integration	A new authentication endpoint (`/auth/google/`) allows users to sign in using their Google account. The system securely verifies the Google ID token and exchanges it for a native API token, creating a new user profile if one does not already exist.
Personalized News Feed	A `/feed/` endpoint was developed to provide users with a news feed. It supports a global view of all posts and a personalized view that shows posts only from users they follow, controlled via a URL query parameter.
Pagination	To ensure performance and scalability, API endpoints that return lists (news feed and post comments) now use pagination. The client can control the number of items per page, and the API provides links to subsequent pages.

1.3. Milestone 1 vs Milestone 2 Comparison

1. **Complexity of Relationships**
    - **Milestone 1:** Focused on core one-to-many relationships (User→Post, User→Comment, Post→Comment) and basic CRUD behavior.
    - **Milestone 2:** Adds richer relational behavior:
        - Likes (`User ↔ Post`) with duplicate prevention enforced both at the database level and in view logic.
        - Follow relationships (`User ↔ User`) for scoped feed retrieval.

    - Post-scoped comments with explicit newest-first ordering and pagination for scalable retrieval.
      These are implemented via `Like/Follow` models, unique constraints, and post-scoped comment endpoints.

2. **Authentication Evolution**
    - **Milestone 1:** Primarily centered on DRF token authentication endpoints and middleware.
    - **Milestone 2:** Keeps token auth but extends authentication with **Google OAuth ID-token exchange** (`/posts/auth/google/`), then issues/returns API token for app use.
      So this is an **evolution (hybrid model)**, not a full replacement of token auth.

3. **Data Management and Scalability**
    - **Milestone 1:** Basic list endpoints with unpaginated retrieval patterns.
    - **Milestone 2:** Introduces production-oriented retrieval patterns:
        - **Pagination** (`PageNumberPagination`) for comment and feed endpoints.
        - **Personalized feed filtering** (`?filter=following`) and global feed support.
        - Query optimization via `select_related` + `prefetch_related` for feed composition.
          This significantly improves behavior under larger data volume compared to flat list returns.

4. **Architectural Rigor (Design Patterns)**
    - **Milestone 1:** Mostly standard DRF structure.
    - **Milestone 2:** Explicitly applies:
        - Factory Pattern via `PostFactory` to centralize post creation behavior.
        - Singleton Pattern via `LoggerSingleton` for centralized logging instance reuse.
          These are concretely implemented in project modules and used in views.

5. **Documentation and Visualization**

- **Milestone 1:** Mostly textual endpoint-focused documentation.
- **Milestone 2:** Documentation now includes architecture and flow visualization artifacts:
    - ERD coverage (including Like relationships),
    - API flow diagrams,
    - Google OAuth flow references.
      This gives clearer system-level understanding beyond endpoint lists alone.

2. System Architecture Update

2.1. Updated Architecture Description

- **Models:**
    - `Like` is clearly a child/join entity tied to exactly one `User` and one `Post`.
    - Duplicate likes are prevented at two layers:
        - DB constraint: unique (`user, post`).
        - View guard: checks existing like before insert.
    - `Comment` is also child-scoped to `Post` and `User`, but in this repository history it already existed in earlier milestone code; Milestone 2 expands behavior around it (post-scoped endpoints + ordering/pagination).

- **Serializers:**
    - Serializers still cover core `User`/`Post` data.
    - `PostSerializer` now surfaces nested comments plus `like_count` and `comment_count`, which directly supports richer feed payloads.
    - Validation exists for non-empty comments in both general and post-scoped comment creation flows (`validate_text` / `validate_content`).

- **Views:**
    - Core `/posts/` and `/posts/{id}/` endpoints remain.
    - Social actions were added as dedicated endpoints:

        - `/posts/{id}/like/`
        - `/posts/{id}/comment/`
        - `/posts/{id}/comments/`
        - `/feed/`
    - `IsPostAuthor` is applied on `PostDetail` object operations.
    - Pagination is implemented in:
        - post-scoped comment list (`PostCommentList`)
        - feed (`FeedView`)
          But `PostListCreate.get` remains unpaginated right now.
    - Pagination and scoped retrieval reduce payload size and improve scalability for high-volume feeds/comments.

- **Authentication:**
    - Token auth is still foundational globally (`DEFAULT_AUTHENTICATION_CLASSES` includes `TokenAuthentication`).
    - Google OAuth was added as an identity bridge:
        - Accepts Google `id_token`
        - Verifies token with Google
        - maps/creates local user
        - returns local API token for the rest of API usage
    - So this is an extension of MS1 auth, not a replacement.

2.2. Updated Diagrams

To reflect the architectural changes, the following diagrams have been created and included with the project documentation:

- **ER Diagram**

This diagram has been updated to include the new Like, Comment, and Follow models and illustrates their relationships with the existing User and Post models.

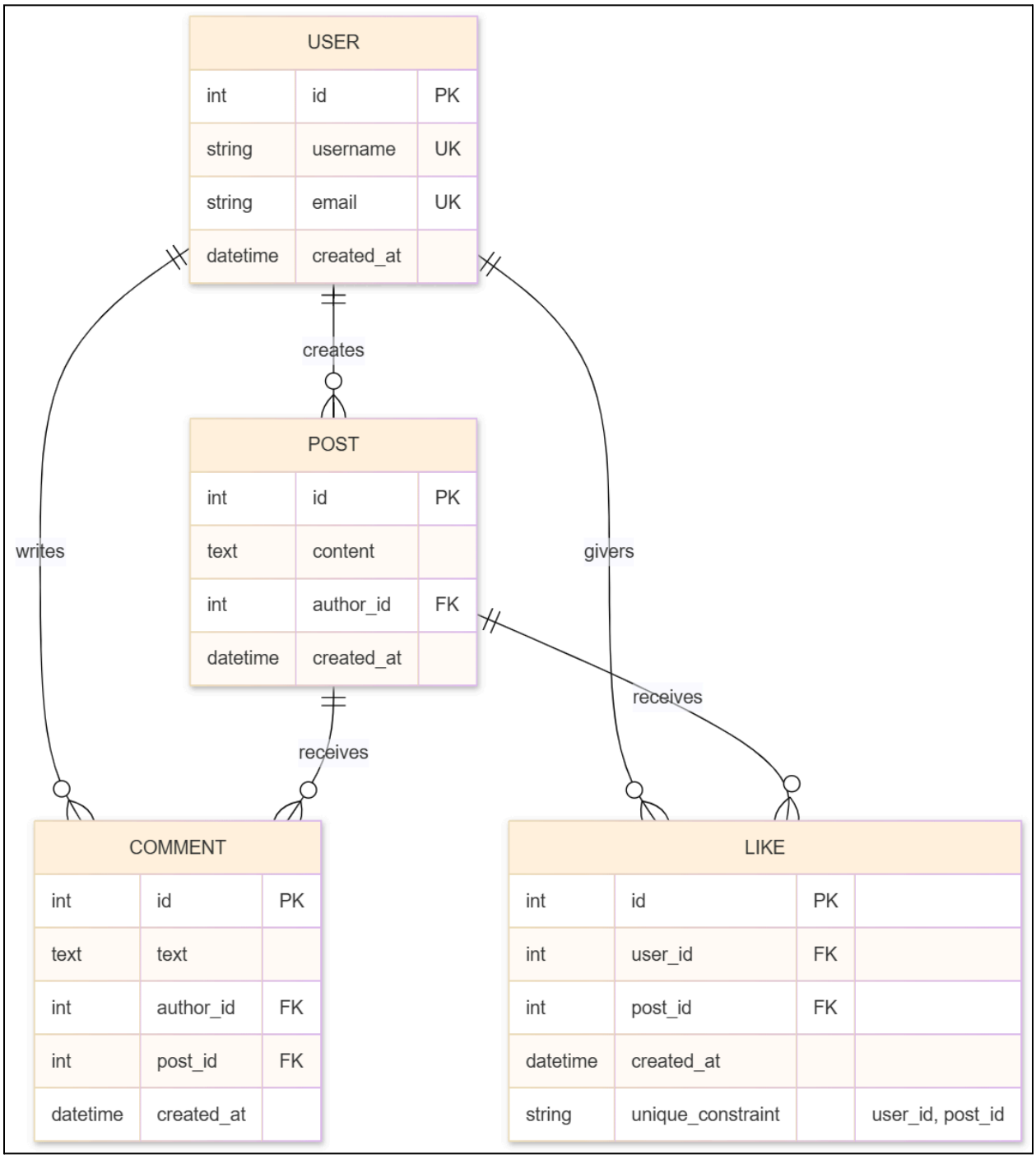

Figure 1: Updated ER Diagram

- **API Flow Diagram**

This diagram visualizes the request/response cycle for the new API endpoints, showing how a client request flows through the authentication, permission, view, and serializer layers to interact with the database.

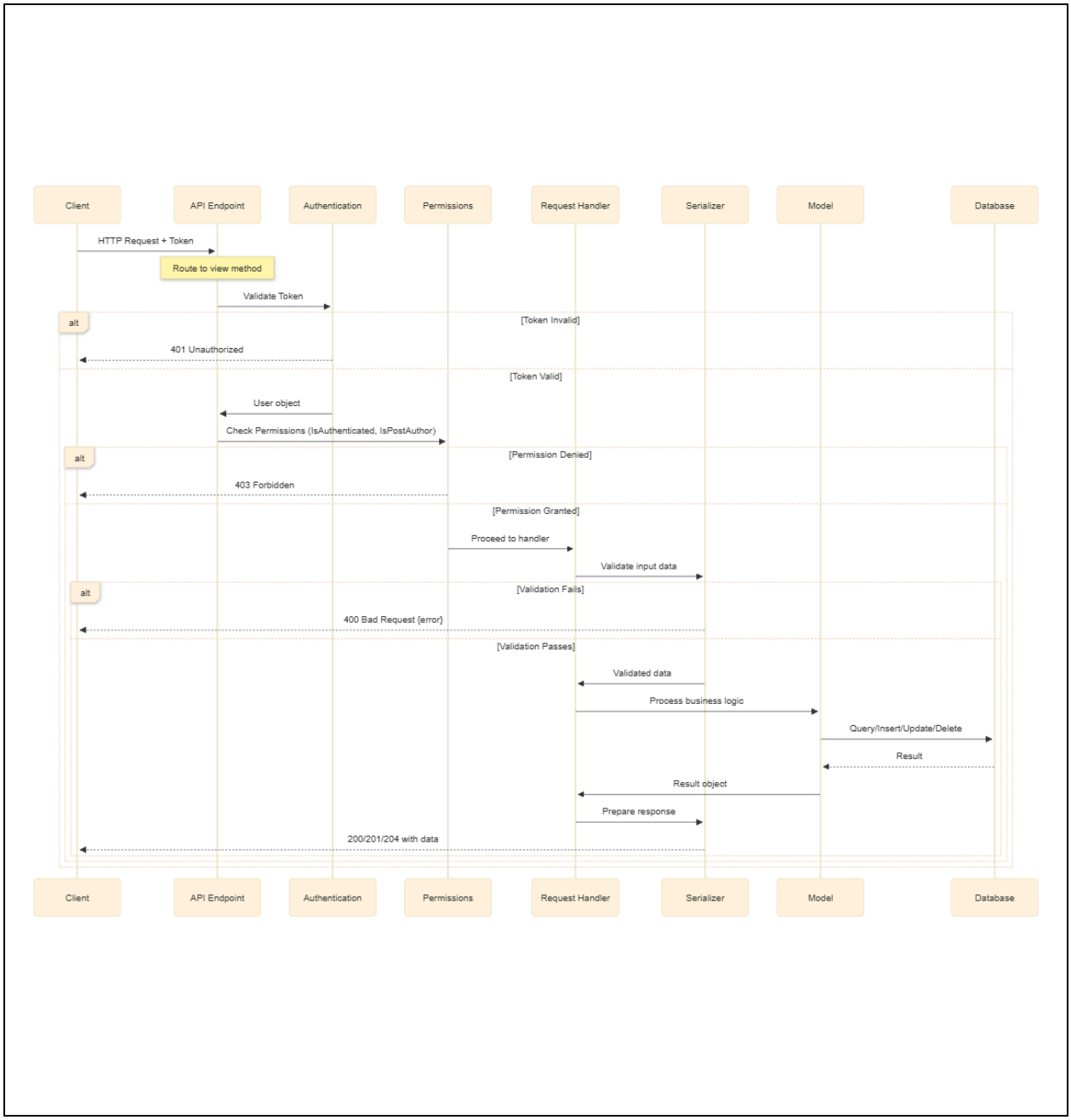

Figure 2: Updated API Flow Diagram

- **OAuth Authentication Flow Diagram**

  A dedicated sequence diagram illustrates the Google login process, from the client sending a Google ID token to the server verifying it and returning a session token.

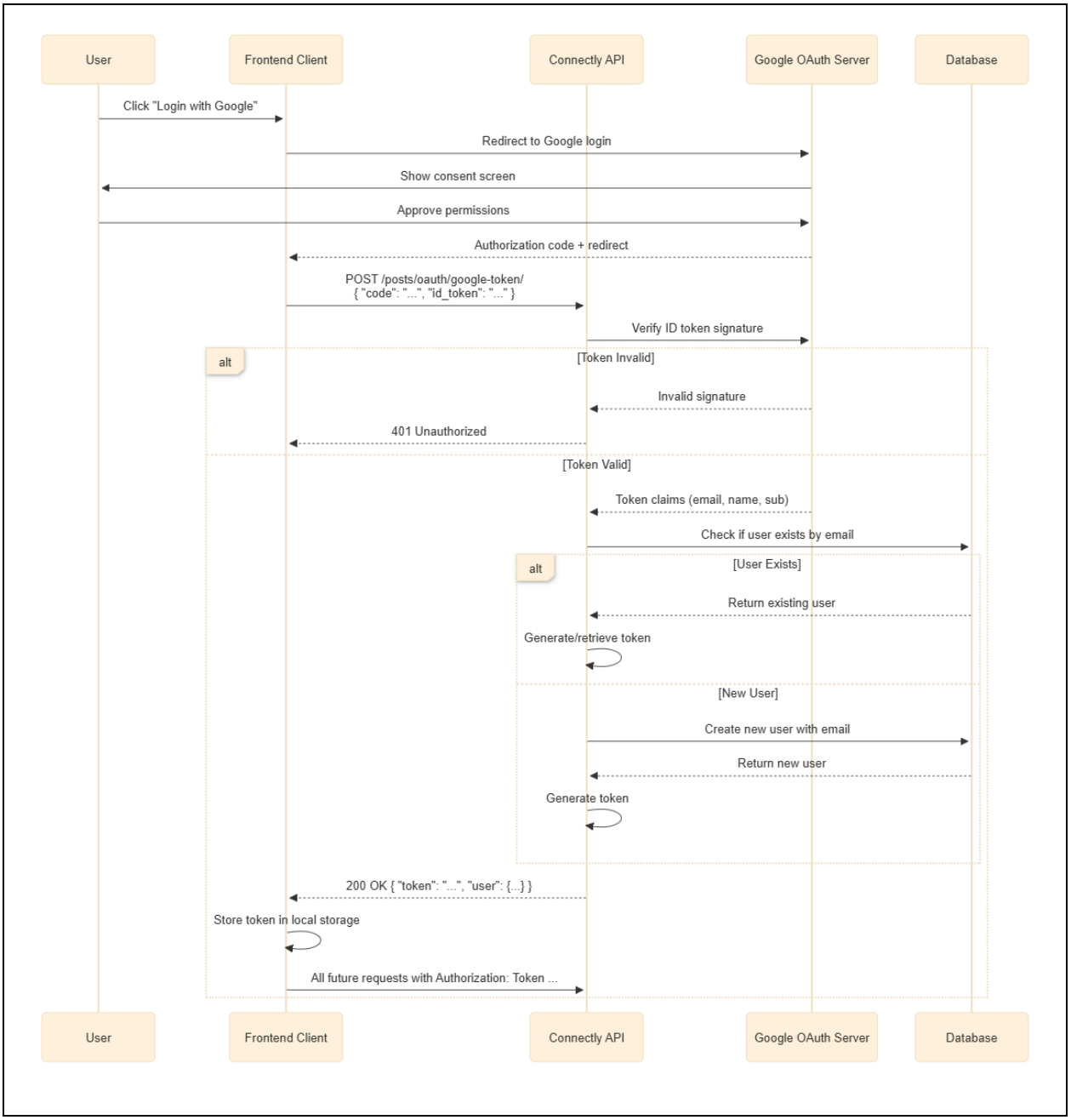

Figure 3: Updated OAuth Authentication Flow Diagram

3. Data Model Changes

3.1. New Models

Three new models were introduced to support the new features:

Model Name	Purpose	Fields	Relationships/Rules
Like	Stores user likes for posts	`id, user, post, created_at`	`user` → FK to `User`; `post` → FK to `Post`; one like per user per post (unique constraint + duplicate check in view).
Comment (post-scoped behavior in MS2)	Enables comments tied to a specific post	**Model fields:** `id, text, author, post, created_at`; **Endpoints:** `POST /posts/{id}/comment/`, `GET /posts/{id}/comments/`	Many-to-one to `User` (author) and many-to-one to `Post`; listing is ordered newest-first and paginated. *(Note: not true threaded comments in current model).
Follow	Stores user-to-user following for personalized feed	`id, follower, followed, created_at`	`follower` and `followed` both FK to `User`; one follow pair only (`unique_together`).
News Feed (MS2 endpoint)	Provides global or following-based feed retrieval	`GET /feed/` (+ optional `?filter=following`)	It uses follow graph to scope posts; feed is paginated and query-optimized.
Google OAuth (MS2 auth addition)	Lets users sign in via Google and receive app token	`POST /auth/google/` with `id_token`	Verifies Google token, maps/creates local user, returns Connectly API token;

			TokenAuthentication remains active globally.

3.2. Model Relationships

The new models establish several key relationships within the database schema:

- A `User` can have many `Comments`, and a `Post` can have many `Comments` (One-to-Many).
- A `User` can have many `Likes`, and a `Post` can have many `Likes` (One-to-Many).
- The `Follow` model establishes a many-to-many relationship on the User model itself, where one user can follow many other users, and can in turn be followed by many others. This is implemented with two foreign keys to the `User` model, distinguished by the `related_name` attributes `following` and `followers`.

4. API Endpoint Documentation

Endpoint Name	URL	Method	Required: Yes / No	Description
Like Post	`/posts/<id>/like/`	POST	Yes (Token)	Toggles a "Like" status for the authenticated user on a specific post.
Unlike Post	`/posts/<id>/like/`	DELETE	—	
Create Comment	`/posts/<id>/comment/`	POST	Yes (Token)	Adds a text comment to the specified post.
List Comments	`/posts/<id>`	GET	Yes (Token)	Retrieves a

	/comments/			paginated list of all comments for a post.
Google Login	/auth/google/	POST	No (ID Token only)	Exchanges a Google ID Token for a Connectly API Token.
News Feed	/feed/	GET	Yes (Token)	Returns posts. Use ?filter=following for a personalized feed.

5. Google OAuth Authentication

5.1. Authentication Flow

The Google OAuth 2.0 integration provides a seamless and secure way for users to authenticate. The flow is as follows:

1. The client application (e.g., a web or mobile app) uses the Google Sign-In library to obtain an `id_token` for the user.
2. The client sends this `id_token` in a `POST` request to the `/posts/auth/google/` endpoint.
3. The `GoogleLogin` view on the server receives the token. It calls `id_token.verify_oauth2_token`, which communicates with Google's servers to validate the token's signature, expiration, and audience (ensuring it was issued for our application's Client ID).
4. Upon successful verification, the server extracts the user's email address.
5. It then checks if a user with that email exists in the Django `AuthUser` table. If the user exists, it retrieves them. If not, it creates a new `AuthUser` with the email and a unique, randomly generated username and secure password.
6. The system ensures a corresponding domain profile (`posts.User`) exists.

7. Finally, the server generates or retrieves a DRF API token (`rest_framework.authtoken.models.Token`) associated with the `AuthUser` and returns it to the client. The client then uses this token for all subsequent authenticated requests.

5.2. Security Controls

Several security measures are in place to protect the authentication process. The primary control is the server-side verification of the Google ID token. This step is critical as it prevents forged tokens from being used. The verification function checks that the token is correctly signed by Google and that its `aud` (audience) claim matches the `GOOGLE_CLIENT_ID` stored in the Django settings, confirming the token was intended for this application. Furthermore, for new users created via this flow, a strong, randomly generated password is assigned, ensuring the account is secure even though it is not used for direct login.

6. News Feed and Pagination

6.1. Sorting Logic

The news feed is designed to be flexible and efficient. The `FeedView` determines which posts to display based on the filter query parameter.

- **Global Feed (Default):** If no filter is provided, the feed returns all posts from all users in the system (`Post.objects.all()`).
- **Following Feed:** If `?filter=following` is included in the request URL, the view first identifies all users that the authenticated user is following. It then filters the posts to include only those authored by this set of followed users.

In both cases, the final queryset of posts is sorted in reverse chronological order (`order_by('-created_at')`), so the newest posts always appear first.

6.2. Pagination Configuration

To handle potentially large numbers of posts and comments efficiently, API endpoints returning lists are paginated using Django REST Framework's PageNumberPagination. The configuration is set within the views:

- **Default Page Size:** If the client does not specify, the API returns 10 items per page.
- **Client-Controlled Page Size:** The client can request a different number of items by including the `page_size` query parameter (e.g., `?page_size=20`).
- **Maximum Page Size:** To prevent abuse, the maximum number of items that can be requested in a single page is capped at 100. The paginated response includes the list of results as well as metadata for the total `count` of items and URLs for the `next` and `previous` pages.

7. Validation and Error Handling

The system employs a multi-layered approach to validation and error handling to ensure data integrity and provide clear feedback to clients. Serializers are the first line of defense, validating the type, format, and constraints of incoming request data. For example, the `PostCommentCreateSerializer` ensures that comment content is not empty.

Business logic validation occurs within the views. For instance, the `PostLike` view checks if a user has already liked a post before attempting to create a new `Like` object, preventing duplicate entries at the application level. When an error is detected, a consistent JSON error payload ({`"error": "A descriptive message"`}) is returned with an appropriate HTTP status code (e.g., 400, 404, 403), facilitated by a shared `_error` helper function.

8. Testing Evidence

8.1. Test Cases

A comprehensive suite of automated tests was written using Django's `APITestCase` to verify the functionality of the new features. Key test cases include:

- **Likes:** Testing that a user can successfully like a post, that attempting to like the same post twice results in a `400 Bad Request` error, and that liking a non-existent post returns a `404 Not Found` error.

- **Comments:** Verifying that a user can comment on a post, that an empty comment is rejected, and that comments are correctly associated with the post and author.
- **Google Login:** Using mock objects to simulate Google's token verification, tests confirm that a new user is created correctly, an existing user can log in, an invalid token is rejected, and a missing token is handled gracefully.
- **News Feed:** Testing that the global feed returns all posts and that the `?filter=following` parameter correctly filters the feed to show only posts from followed users.

8.2. Screenshots W6-9 Test Evidence

Manual testing was performed using Postman to complement the automated tests. Screenshots of these test runs serve as evidence of the API's correct behavior in a real-world client scenario. These screenshots document:

- Successful `201 Created` responses when liking a post or creating a comment.
- `400 Bad Request` error responses when attempting to like a post twice.
- The paginated structure of responses from the `/feed/` and `/comments/` endpoints, showing the `count`, `next`, `previous`, and `results` fields.
- Successful retrieval of an API token from the `/auth/google/` endpoint after providing a valid (mock) ID token.

9. Improvements From Milestone 1

This milestone builds upon the foundational CRUD functionality of Milestone 1 by introducing a layer of dynamic social interaction. While Milestone 1 established the core entities of Users and Posts, Milestone 2 makes the system feel like a true social network. The addition of Google OAuth provides a more modern and secure authentication alternative to the basic username/password flow. The news feed and pagination features address critical user experience and performance considerations, transforming the system from a simple data repository into a scalable, interactive platform.

10. Challenges and Solutions

During development, several challenges were encountered and overcome:

Challenge and Solution 1:

Challenge: Initial failures in Google token verification within the local development environment, presenting a `Certificate for key id not found` error.

Solution: Investigation revealed this was not a code issue but an environmental one. The local Django server was unable to make an outbound HTTPS request to Google's servers to fetch the public keys for token verification. The solution was to ensure the development machine had an active and unrestricted internet connection, resolving the issue immediately.

Challenge and Solution 2:

Challenge: Ensuring that Django's built-in `User` model (for authentication) and the application's custom `User` model (for domain logic) remained synchronized, especially when users were created through non-standard means like Google login or the `createsuperuser` command.

Solution: A helper function, `_get_domain_user`, was implemented. This function is called in views after authentication and transparently creates a domain User profile if one does not already exist for the authenticated user. This defensive approach makes the system more resilient and decouples user creation logic from individual endpoints.

Challenge and Solution 3:

Challenge: Designing the news feed to be performant and avoid excessive database queries, which could slow down the API as the number of users and posts grows.

Solution: The Django ORM's query optimization tools were leveraged. In the `FeedView`, `select_related('author')` is used to fetch the author of each post with a single `JOIN`, and `prefetch_related('likes', 'comments')` is used to fetch all likes and comments for the retrieved posts in separate,

minimal queries. This strategy, known as eager loading, significantly reduces the database load compared to fetching related objects for each post individually.

11. Conclusion

Milestone 2 successfully extended the Connectly backend with a suite of essential social features. The implementation of likes, comments, Google OAuth, and a personalized news feed has transformed the system into a viable platform for a modern social application. By overcoming challenges related to external service integration, data model synchronization, and database performance, the project has demonstrated a commitment to robust, scalable, and secure software engineering principles. The resulting API is well-documented, thoroughly tested, and prepared for future expansion.